

서민재

BE Engineering · Reliability · Search & Data · AI Engineering

Java와 Kotlin, Spring으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 왔습니다. **실패 지점과 재시작 기준**을 코드에 남기는 백엔드 설계를 바탕으로, 코드 탐색·설계 검토·테스트를 **에이전트와 스킬**로 구조화합니다.

ckdekrn88@gmail.com

github.com/ch4570

velog.io/@ch4570

linkedin.com/in/devseorex

이력서

포트폴리오

Seoul, Korea

경력 요약

01 · RELIABILITY

실패 지점과 재시작 기준을 설계합니다.

이벤트 상태, 복구 전략, 배치 순서를 코드와 테스트로 관리했습니다.

02 · SEARCH & DATA

검색·랭킹·캐시의 경계를 먼저 세웁니다.

피드 전 과정을 단계별 계약으로 나누고 의존 방향을 테스트로 고정했습니다.

03 · AI-NATIVE

반복한 개발 절차를 다른 저장소에 설치 가능한 자산으로 만듭니다.

에이전트·스킬·평가 계약을 manifest로 연결하고, 변경 전 사람 검토를 요구하는 계약을 정의했습니다.

2025.06 — 현재	웍스피어(유)	JOBKOREA / Albamon Platform · Backend Engineer
2024.06 — 2025.04	토스랩	JANDI · Backend Engineer
2023.09 — 2024.05	오케스트로	IDP / Vista · Backend Engineer
2022.11 — 2023.02	즐거운	Backend Engineer

백엔드 운영과 AI 개발에 같은 판단 순서를 적용합니다.

관찰한 문제를 경계로 바꾸고, 실행 결과로 다시 확인합니다.

01 · 관찰

문제가 드러나는 지점을 찾습니다.

실패 위치 · 조건 변화 · 반복 설정 · 실행 근거

구체화

02 · 경계 설계

다시 시작할 기준을 코드로 남깁니다.

상태 · 재시작 · 캐시 무효화 · 승인 계약

실행

03 · 검증

끝난 뒤의 상태로 판단합니다.

운영 신호 · 결과 불변식 · 회귀 · 평가 계약

운영 신호

정제 이벤트 집계

동시성

API별 결과 불변식

설치 회귀

설치 자산 누락 검사

평가 계약

구조·행동 계약 검사

웍스피어(유) · Backend Engineer

JOBKOREA·Albamon Platform의 이벤트 신뢰성, 배치, 검색·캐시, 공통 테스트 기반을 설계·구현합니다.

2025.06 — 현재

POINT PLATFORM · EVENT RELIABILITY

이벤트 실패 지점을 6개 상태로 남기고 8개 복구 전략을 연결했습니다.

발행 실패와 소비 중 실패는 다시 시작할 위치가 다르므로 처리 단계 자체를 이벤트 레코드에 남겼습니다.

- 내부 이벤트 9종·6개 상태로 생성부터 복구 불가 실패까지 기록했습니다.
- 8개 수동 복구 전략으로 저장된 payload를 이벤트 유형별 처리 경로에 전달했습니다.
- 15분 정체 알림으로 오래 멈춘 이벤트를 유형별로 모아 Slack에 알렸습니다.

운영 기준 전송 실패, 처리 실패, 장기 미처리를 구분해 멈춘 위치부터 복구합니다.

POINT LEDGER · BATCH / ARCHIVE

배치는 다시 시작할 위치를, 분산락은 경쟁 후 상태를 테스트로 고정했습니다.

배치는 멈춘 위치를, 동시 요청은 처리가 끝난 뒤의 잔액과 상태를 확인했습니다.

- 페이지 기반 Chunk 처리로 포인트 소멸의 조회와 쓰기 단위를 나눴습니다.
- archive 테이블 10개의 이관 순서와 재시작 기준을 테스트로 고정했습니다.
- 9개 API·13개 동시성 시나리오에서 잔액·원장 상태·후속 이벤트를 검증했습니다.

웍스피어 · BACKEND ENGINEERING

검색·캐시와 테스트 기반

FEED & SEARCH · SERVING / RETRIEVAL

후보 수집부터 노출까지, 개인화 피드의 온라인 서빙 구조를 세웠습니다.

검색·랭킹·캐시를 포트와 단계별 계약으로 나눴습니다. 오케스트레이터는 캐시 적중·소진·장애에 따라 서빙 흐름을 조정하고, 각 랭킹 단계는 자기 계산을, FPR·SPR는 후보 풀 캐시까지 말합니다.

- 9개 검색 소스의 필터·정렬 규칙과 사용할 벡터를 소스별로 나누고, OpenSearch 호출은 공통 어댑터 뒤로 감쌌습니다. 한 소스가 늦거나 실패해도 나머지 후보는 유지합니다.
- FPR·SPR·Re-Rank를 하나의 pull 계약으로 연결하고 Snapshot·FPR·SPR 3종 캐시의 적재·소비 책임을 단계별로 나눴습니다.
- query-api 모듈은 OpenSearch 연동만 말도록 격리하고, ArchUnit으로 모듈 의존 방향과 순환 금지를 검증했습니다.

설계 기준 후보 수보다 검색 소스, 랭킹 정책, 캐시 장애가 바뀔 때 수정 범위가 어디까지 번지는지를 먼저 봤습니다.

ENGINEERING PLATFORM · TEST SUPPORT

공통 모듈과 사용 규칙으로 테스트 진입점을 정리했습니다.

반복 설정을 공통화하고, 테스트 선택 기준과 CI 실행 범위를 문서로 남겼습니다.

- 구축 당시 29개 구현 파일·43개 테스트 소스로 fixture·mock·WebMvc·JPA·트랜잭션 공통 동작을 묶었습니다.
- 4개 진입점·5개 공통 도구와 정책으로 테스트 목적에 맞는 준비 코드와 실행 기준을 제공했습니다.
- 테스트 작성 원칙을 제안하고 느린 동시성 테스트를 skip.slow.test 설정으로 분리했습니다.

Agent Lab · 개발 판단을 설치·평가 가능한 자산으로 만들었습니다.

코드 탐색·설계 검토·테스트·리뷰 절차를 다른 프로젝트에서도 다시 쓸 수 있는 단위로 정리했습니다.

V0.1.0.0

역할·지식·절차·평가를 나눠 manifest로 연결했습니다.

- 15개 에이전트·14개 지식 번들·32개 스킬과 24개 연결, 11개 의존성을 선언했습니다.
- 설치 회귀 52/52·Codex 탐색 32/32로 자산과 의존성이 공식 경로에 설치되는지 확인했습니다.
- 구조 7/7·행동 계약 10/10을 정적으로 검증하고 사람이 승인해야 다음 설치에 반영되도록 계약했습니다.

현재 경계 실행 런타임이 아닌 이식 가능한 자산 저장소입니다. 장기 메모리, 프롬프트 자동 수정, 스킬 자동 승격은 포함하지 않았습니다.

PREVIOUS ROLE

토스랩 · JANDI Backend Engineer

2024.06 — 2025.04

커버리지 70%에도 배포가 불안해 실제 구현 검증과 CI 재현 조건부터 다시 봤습니다.

- Spring Boot 2.3.8 → 3.3.3 전환과 함께 테스트를 0개에서 160개 이상으로 늘렸습니다.
- 약 200만 건 데이터 보정을 위해 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다.
- 자원 증설 없이 CI 실행·컨텍스트 재사용·초기화 구조를 바꿔 메모리 부족을 해결했습니다.

이전 경력

오케스트로 · Backend Engineer

IDP / Vista

2023.09 — 2024.05

- API Gateway·Security·JWT 기반 인증·인가와 SonarQube·Jenkins 연동 API를 개발했습니다.
- VM 메트릭·OpenSearch 조회 API와 조건별 동적 쿼리를 구현했습니다.

즐거운 · Backend Engineer

자사 서비스 개발·운영

2022.11 — 2023.02

자사 백엔드 기능을 개발하고 운영 이슈를 대응했습니다.